

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted

Nimay Anand Desai (1WA24CS189)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING

in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by NimayAnand Desai (1WA24CS189), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil

Assistant Professor

Department of CSE

BMSCE, Bengaluru

Dr. Kavitha
Sooda

Professor and
Head

Department of
CSE

B M S C E ,
Bengaluru

Index Sheet

Sl No	Experiment Title	Page No
1	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-35
2	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	36-44
3	Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one) a) Rate-Monotonic b) Earliest-Deadline First c) Proportional Scheduling	45-61
4	Write a C program to simulate: (Any one) a) Producer-Consumer problem using semaphores b) Dining-Philosopher's problem	62-75
5	Write a C program to simulate: (Any one) a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock Detection	76-94
6	Write a C program to simulate the following contiguous memory allocation techniques. (Any one) a) Worst-Fit b) Best-Fit c) First-Fit	95-105
7	Write a C program to simulate page replacement algorithms. (Any one) a) FIFO b) LRU c) Optimal	106-119

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS
b) SJF
c) Priority
d) Round Robin (Experiment with different quantum sizes for RR algorithm)

FCFS

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: Nimay Anand Desai\nUSN:1WA24CS189\n\n"); int n, i;
printf("Enter number of processes: ");
scanf("%d", &n);

int at[n], bt[n], ct[n], tat[n], wt[n];

printf("Enter Arrival Time:\n");
for(i = 0; i < n; i++)
scanf("%d", &at[i]);

printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
scanf("%d", &bt[i]);

int t = 0;
for(i = 0; i < n; i++)
{
if(t < at[i])
t = at[i];

ct[i] = t + bt[i];
t = ct[i];
}
```



```

    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i]; }

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n"); for(i
= 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Name: Nimay Anand Desai

USN:1WA24CS189

Enter number of processes: 4

Enter Arrival Time:

0 0 0 0

Enter Burst Time:

7 3 4 6

Process	AT	BT	CT	TAT	WT
P1	0	7	7	7	0
P2	0	3	10	10	7
P3	0	4	14	14	10
P4	0	6	20	20	14

PS C:\Users\Usere\Documents\LAB_REPORT>

2

Program-2FCS

→ #include <stdio.h>

int main() {
int n, i;printf("Enter total number of processes:");
scanf("%d", &n);

int at[n], bt[n], ct[n], tat[n], wt[n];

float avg-tat=0, avg-wt=0;

for (i=0; i<n; i++) {

printf("\n process [%d] \n", i+1);

printf("Arrival time:");
scanf("%d", &at[i]);printf("Burst time:");
scanf("%d", &bt[i]);

}

ct[0] = at[0] + bt[0];

for (i=1; i<n; i++) {

if (ct[i-1] < at[i])

ct[i] = ct[i-1] + bt[i];

else

ct[i] = ct[i-1] + bt[i];

}

3

for

3

```
for (i=0; i<n; i++) {
```

```
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
```

```
    avg_tat += tat[i];
    avg_wt += wt[i];
```

```
}
```

```
avg_tat = avg_tat / n;
avg_wt = avg_wt / n;
```

```
printf("\n Process \t \t Arrival time \t \t Burst  
time \t \t completion time \t \t turnaround  
time \t \t waiting time\n");
```

```
for (i=0; i<n; i++) {
```

```
    printf("%d \t %d \t %d \t %d \t %d \t  
          %d \t %d \t %d \t %d\n",
```

```
          i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
```

```
}
```

```
printf("\n Average Turnaround Time: %2f",  
       avg_tat);
```

```
printf("\n Average waiting Time: %2f",  
       avg_wt);
```

```
return 0;
```

```
}
```


Q/P1-

Enter no. of process = 4

Enter process id, arrival time, Burst time, for 4 processes.

process 1: 10 8

process 2: 2 3 9

process 3: 3 5 7

process 4: 4 9 4

Process scheduling - FCFS algorithm.

PID	AT	BT	CT	TAT	WT
1	6	8	8	8	6
2	3	9	12	14	5
3	5	7	24	19	12
4	9	4	28	14	15

avg waiting time : 8

avg Turn around time : 15

Qva

SJF (Non – Preemptive)

CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], ct[n], tat[n], wt[n];
    int done[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
    { scanf("%d", &at[i]);
      done[i] = 0; }

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    int t = 0, completed = 0;

    while(completed < n)
    { int min = 9999;
      int index = -1;

      for(i = 0; i < n; i++)
      { if(at[i] <= t && done[i] == 0)
        { if(bt[i] < min)
          { min = bt[i];
            index = i;
          }
        }
      }
    } }
```



```

    if(index == -1)
    { t++;
    }
    else
    { ct[index] = t + bt[index];
      t = ct[index];

      tat[index] = ct[index] - at[index];
      wt[index] = tat[index] - bt[index];

      done[index] = 1; completed+
      +;
    } }

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n"); for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
  i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

Name: Nimay Anand Desai
USN:1WA24CS189

Enter number of processes: 4
Enter Arrival Time:
0 0 0 0
Enter Burst Time:
7 3 4 6

Process	AT	BT	CT	TAT	WT
P1	0	7	20	20	13
P2	0	3	3	3	0
P3	0	4	7	7	3
P4	0	6	13	13	7

PS C:\Users\Usere\Documents\LAB_REPORT> █

Program-3Algorithm SJF. non-preemptive:-

```

→ #include <stdio.h>
int main () {
    int n, i;
    int pid [10], arrival [10], burst [10], ct [10], tat [10],
        , utt [10];
    int done [10] = {0};
    float avg-wt = 0, avg-tat = 0;

    printf ("Enter number of processes ");
    scanf ("%d", &n);
    for (i=0; i < n; i++) {
        printf ("%d, PID at Bt = ", i+1);
        scanf ("%d %d %d", &pid[i], &arrival[i], &burst[i]);
    }

    int time = 0;
    while (1) {
        int min = 999, pos = -1;
        for (i=0; i < n; i++) {
            if (!done[i] & arrival[i] <= time & burst[i] < min) &
                min = burst[i];
            pos = i;
        }
        if (pos == -1) break;
        done[pos] = 1;
        tat[pos] = time - arrival[pos];
        utt[pos] = tat[pos] + burst[pos];
        time = time + burst[pos];
    }

    float avg-tat = 0, avg-utt = 0;
    for (i=0; i < n; i++) {
        avg-tat += tat[i];
        avg-utt += utt[i];
    }
    avg-tat /= n;
    avg-utt /= n;

    printf ("Average tat = %f\n", avg-tat);
    printf ("Average utt = %f\n", avg-utt);
}

```


6

```

if (pos == -1) {
    time = 0;
    int all_done = 1;
    for (i = 0; i < n; i++) if (!done[i])
        if (all_done) break;
    }
    at[pos] = time + burst[pos];
    tat[pos] = ct[pos] + arrival[pos];
    wt[pos] = tat[pos] - burst[pos];
    done[pos] = 1;
    time = ct[pos];
}
}

```

```

printf("AT BT CT TAT WT \n");
for (i = 0; i < n; i++) {
    printf("%d, %d, %d, %d, %d, %d, \n");
    pid[i], arrival[i], burst[i], ct[i], tat[i], wt[i];
    avg_wt = wt[i];
    avg_tat = tat[i];
}

```

```

printf("%d", wt[i]);
printf("%d", avg_tat/n);

```

```

return 0;
}

```

O/P:-

enter no of process = 4

P₁ : 106

P₂ : 224

P₃ : 342

P₄ : 468

7

PID

1

2

3

4

a

a

7

PID	AT	BT	CT	TAT	WT
1	0	1	6	6	0
2	2	4	12	10	6
3	4	2	8	2	2
4	6	8	10	14	6

$$\text{avg WT} = 3.5$$

$$\text{avg TAT} = 8.5$$

SRTF

CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
    { scanf("%d", &bt[i]);
      rt[i] = bt[i]; }

    int t = 0, completed = 0;

    while(completed < n)
    { int min = 9999;
      int index = -1;

      for(i = 0; i < n; i++)
      { if(at[i] <= t && rt[i] > 0)
        { if(rt[i] < min)
          { min = rt[i];
            index = i;
          }
        }
      }
    } }
```



```

    if(index == -1)
    { t++;
    }
    else
    {
        rt[index]--; t+
        +;

        if(rt[index] == 0)
        { ct[index] = t;
          completed+
          +;

          tat[index] = ct[index] - at[index];
          wt[index] = tat[index] - bt[index];
        }
    }
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Result:

Result:

Name: Nimay Anand Desai

USN:1WA24CS189

Enter number of processes: 4

Enter Arrival Time:

0 0 0 0

Enter Burst Time:

7 3 4 6

Process	AT	BT	CT	TAT	WT
P1	0	7	20	20	13
P2	0	3	3	3	0
P3	0	4	7	7	3
P4	0	6	13	13	7

PS C:\Users\User\Documents\LAB_REPORT> █

8

Program - 4Algorithm SJF - promptile :-

→ # include <stdio.h>

int main() {

int n, i;

int pid[10], arrival[10], burst[100],

remaining[10], int ct[10], tat[10], et[10];

int done[10] = {0};

float avg-wt = 0, avg-tat = 0;

printf("Enter no. of processes: ");

scanf("%d", &n);

for (i=0; i<n; i++) {

printf("%d", &id AT BT= ^; i+1);

scanf("%d %d %d", &apid[i], &arrival[i], &burst[i]);

remaining[i] = burst[i];

}

int time = 0;

while (1) {

int all_done = 1;

for (i=0; i<n; i++) {

if (!done[i] all_done = 0; }

}

if (all_done) break;

int min = 999, pos = -1;

for (i=0; i<n; i++) {

if (done[i] && arrival[i] <= time && remaining[i] < min) {

min :

pu

{

if

if

a

```

min = remaining [i];
pos = i;
}
}
if (pos == -1) {
time ++;
if (remaining [pos] == 0) {
ct [pos] = time;
tat [pos] = ct [pos] - arrival [pos];
wt [pos] = tat [pos] - hw [pos];
done [pos] = 1;
}
}
}

printf ("Avg_wt : %2t", avg_wt (n));
printf ("Avg_tat : %2t", avg_tat (n));
printf ("Avg_tat : %2t", avg_tat (n));
return 0;
}

```

O/P:-

Enter Process : 4

PID	AT	BT	CT	TAT	WT
1	0	6	6	6	0
2	2	4	15	13	9
3	4	2	8	4	2
4	6	3	11	5	2

$$\text{avg WT} = 3.25$$

$$\text{avg TAT} = 2$$

$$\text{avg TAT} = 2$$

PRIORITY NON-PREEMPTIVE

CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n];
    int done[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
    { scanf("%d", &at[i]);
      done[i] = 0; }

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &bt[i]);

    printf("Enter Priority:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    int t = 0, completed = 0;
    while(completed < n)
    { int min = 9999;
      int index = -1;

      for(i = 0; i < n; i++)
      { if(at[i] <= t && done[i] == 0)
        { if(pr[i] < min)
```



```

        { min = pr[i];
          index = i;
        }
      }
    }

    if(index == -1)
    { t++;
    }
    else
    { ct[index] = t + bt[index];
      t = ct[index];

      tat[index] = ct[index] - at[index];
      wt[index] = tat[index] - bt[index];

      done[index] = 1; completed+
      +;
    }
  }

  printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

  for(i = 0; i < n; i++)
  { printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
  }

  return 0;
}

```

Result:

Result:

Name: Nimay Anand Desai
USN:1WA24CS189

Enter number of processes: 5

Enter Arrival Time:

0 2 3 4 5 6

Enter Burst Time:

3 2 5 4 1

Enter Priority:

5 3 2 4 1

Process	AT	BT	PR	CT	TAT	WT
P1	0	6	1	6	6	0
P2	2	3	5	20	18	15
P3	3	2	3	13	10	8
P4	4	5	2	11	7	2
P5	5	4	4	17	12	8

PS C:\Users\Usere\Documents\LAB_REPORT>

Program-5Non-Preemptive Priority Scheduling:-

```

→ #include <stdio.h>
#include <limits.h>
#define max-process 100

int main () {
int n;
int process-id [max-process];
int arrival-time [max-process];
int burst-time [max-process];
int completed [max-process];
    int n;
    int pid [20], at [20], bt [20], pr [20];
    int ct [20], tat [20], at [20];
    int completion [20] = {0};

    int current_time = 0; completed_count = 0;
    int i;

    printf ("Enter the no. of process: ");
    scanf ("%d", &n);

    for (i = 0; i < n; i++) {
        printf ("Process: ");
        printf ("Enter process ID: ");
        scanf ("%d", &pid[i]);

        printf ("Enter AT: ");
        scanf ("%d", &at[i]);
    }

```



```

    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);

    printf("Enter priority: ");
    scanf("%d", &pr[i]);
}

while [completed_count < n] {
    int highest_priority = 9999;
    int selected_process = -1;

    for (i=0; i<n; i++) {
        if (at[i] <= cum && comp[i] == 0) {
            if (pr[i] < highest_priority) {
                highest_priority = pr[i];
                selected_process = i;
            }
        }
    }

    if (selected_process == -1) {
        current_time++;
    }
    else {
        ct[selected_process] = cum + bt[selected_process];
        tat[selected_process] = ct[selected_process] -
            at[selected_process];
        ct[selected_process] = tat[selected_process] - bt[
            selected_process];
        current_time = ct[selected_process];
        completed[selected_process] = 1;
        completed_count++;
    }
}
}

```


12

```

printf ("Process AT BT priority CT TAT WT\n");
for (i=0; i<n; i++) {
    printf ("%d %d %d %d %d %d\n",
            pid[i], at[i], bt[i], pri[i], ct[i],
            tat[i]);
}
return 0;
}

```

Pre-empt
→ #Process
Print ma

O/P:-

Enter num of process = 5
Enter Process ID, AT, BT, Priority:
1 0 3 5

Enter Process ID, AT, BT, Priority:
2 2 2 3

Enter Process ID, AT, BT, Priority:
3 3 5 2

Enter Process ID, AT, BT, Priority:
4 4 4 4

Enter Process ID, AT, BT, Priority:
5 6 1 1

Process	AT	BT	Priority	CT	TAT	WT
P1	0	3	5	3	3	0
P2	2	2	3	11	9	7
P3	3	5	2	8	5	0
P4	4	4	4	15	11	7
P5	6	1	1	9	3	2

PRIORITY PREEMPTIVE
CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], rt[n];
    int ct[n], tat[n], wt[n];

    printf("Enter Arrival Time:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &at[i]);

    printf("Enter Burst Time:\n");
    for(i = 0; i < n; i++)
    { scanf("%d", &bt[i]);
      rt[i] = bt[i]; }

    printf("Enter Priority:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pr[i]);

    int t = 0, completed = 0;
    while(completed < n)
```



```

{ int min = 9999;
  int index = -1;

  for(i = 0; i < n; i++)
  { if(at[i] <= t && rt[i] > 0)
    { if(pr[i] < min)
      { min = pr[i];
        index = i;
      }
    }
  }

  if(index == -1)
  { t++;
  }
  else
  {
    rt[index]--; t+
    +;

    if(rt[index] == 0)
    { ct[index] = t; tat[index] =
      ct[index] - at[index]; wt[index] =
      tat[index] - bt[index];
      completed++;
    }
  }
}

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
  i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```


Result:

Result:

Name: Nimay Anand Desai
USN:1WA24CS189

Enter number of processes: 7

Enter Arrival Time:

6 4 1 3 0 5 10

Enter Burst Time:

5 1 2 4 8 6 1

Enter Priority:

6 5 4 4 3 2 1

Process	AT	BT	PR	CT	TAT	WT
P1	6	5	6	27	21	16
P2	4	1	5	22	18	17
P3	1	2	4	17	16	14
P4	3	4	4	21	18	14
P5	0	8	3	15	15	7
P6	5	6	2	12	7	1
P7	10	1	1	11	1	0

PS C:\Users\Usere\Documents\LAB_REPORT> █

Program - 6

Pre-emptive Priority scheduling:-

```
→ #include <stdio.h>
int main() {
    int n, choice;
    int pid[20], pri[20], at[20], bt[20], rt[20];
    int ct[20], tat[20], wt[20];
    int completed = 0, cur_time = 0;
    int i, set, best;

    float avg_wt, avg_tat = 0;

    printf("Enter no. of process: ");
    scanf("%d", &n);

    printf("Enter priorities:");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pri[i]);
    }

    printf("Enter AT: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &at[i]);
    }

    printf("Enter RT: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
        pid[i] = i + 1;
    }

    while (comp < n) {
        selected = -1;
        if (choice == 2)
```


best-priority = 9999;

else
best-priority = -1;

```
for (i=0; i<n; i++) {
    if (arr[i] <= cum && arr[i] > 0) {
        if (choice == 2) {
            if (arr[i] < best-pr) {
                if (choice == 2) {
                    if (arr[i] < best-pr) {
                        best-pr = arr[i];
                        selected = i;
                    }
                }
            }
        }
    }
}
```

```
else {
    if (arr[i] > best-pr) {
        best-pr = arr[i];
        selected = i;
    }
}
```

```
}
if (selected == -1) {
    cum += 1;
}
```

```
else {
    arr[selected]--;
    cum++;
}
```

```
if (arr[selected] == 0) {
    arr[selected] = cum;
}
```



```

    tat(selected) = ct(select) - at(select);
    dlt(select) = tat(select) - bt(select);

```

```

    avg_tat += tat(selected);
    avg_wt += wt(selected);

```

```

    completed++;

```

```

}

```

```

}

```

```

}

```

```

printf("Priority scheduling (pre-empt) :);

```

```

printf("PID    Pri    AT    BT    CT    TAT    WT    RT");

```

```

for(i=0; i<n; i++) {

```

```

    printf("%d %d %d %d %d %d %d\n",

```

```

        pid[i], pr[i], at[i], bt[i], ct[i], tat[i],
        wt[i]);

```

```

}

```

```

printf("Avg TAT: %6f", avg_tat);

```

```

printf("Avg WT: %6f", avg_wt);

```

```

return 0;

```

```

}

```


OP:-

Enter num of process = 7

Enter priority (1 for higher priority, 2 for lower
number means higher priority): 2.

Enter priorities:

3 4 4 5 2 6 1

Enter AT:

0 1 3 4 5 6 10

Enter BT:

8 2 4 1 6 5 1

Priority scheduling (pre-emptive):

PID	Prion	AT	BT	CT	TAT	WT	RT
P6	6	6	5	27	21	16	16
P4	5	4	1	22	18	17	17
P2	4	1	2	17	16	14	14
P3	4	3	4	21	18	14	14
P1	3	0	8	15	15	7	0
P5	2	5	6	12	7	1	0
P7	1	10	1	11	1	0	0

Avg TAT: 13.714 ms

Avg WT: 9.8571 ms.

ROUND ROBIN

CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, tq, i;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    printf("Enter Time Quantum: ");
    scanf("%d",&tq);

    int pid[n], at[n], bt[n], rt[n];
    int wt[n], tat[n], ct[n],
    resp[n]; int queue[100]; int
    front=0, rear=0;

    for(i=0;i<n;i++)
    { printf("Enter Process ID, Arrival Time, Burst Time:\n");
        scanf("%d %d %d",&pid[i],&at[i],&bt[i]);

        rt[i] = bt[i];
        wt[i] = 0;
        tat[i] = 0;
        resp[i] = -1;
    }
    int current_time = 0;
    int completed = 0;
    int visited[n];

    for(i=0;i<n;i++)
        visited[i] = 0;

    while(completed < n)
```



```

{ for(i=0;i<n;i++)
  { if(at[i] <= current_time && visited[i] == 0)
    { queue[rear++] = i;
      visited[i] = 1;
    } }

  if(front == rear)
  { current_time++;
    continue; }

  int p = queue[front++];

  if(resp[p] == -1) resp[p] =
    current_time - at[p];

  if(rt[p] > tq)
  { current_time += tq;
    rt[p] -= tq;

    for(i=0;i<n;i++)
    { if(at[i] <= current_time && visited[i] == 0)
      { queue[rear++] = i;
        visited[i] = 1;
      } }

    queue[rear++] =
p; }
  else
  { current_time += rt[p];
    rt[p] = 0;

    ct[p] = current_time;
    tat[p] = ct[p] - at[p];
    wt[p] = tat[p] - bt[p];
    completed++;
  } }

```



```

float avg_wt=0, avg_tat=0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)
{ printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
  pid[i],at[i],bt[i],ct[i],tat[i],wt[i],resp[i]);

  avg_wt += wt[i];
avg_tat += tat[i]; }

printf("\nAverage Waiting Time = %.2f",avg_wt/n); printf("\nAverage
Turnaround Time = %.2f",avg_tat/n);

return 0;
}

```

Result:

```

Enter Process ID, Arrival Time, Burst Time:
1 0 5
Enter Process ID, Arrival Time, Burst Time:
2 1 3
Enter Process ID, Arrival Time, Burst Time:
3 2 1
Enter Process ID, Arrival Time, Burst Time:
4 3 2
Enter Process ID, Arrival Time, Burst Time:
5 4 3

PID    AT    BT    CT    TAT    WT    RT
1      0     5    13    13     8     0
2      1     3    12    11     8     1
3      2     1     5     3     2     2
4      3     2     9     6     4     4
5      4     3    14    10     7     5

Average Waiting Time = 5.80
Average Turnaround Time = 8.60

```


2 for lower

21

Program - 7

Round Robin

```
#include <stdio.h>
int main () {
    int n, tq;
    int at[20], bt[20], rt[20];
    int ct[20], tat[20], wt[20];
    int i, time=0, remain, flag=0;

    float avg_wt=0, avg_tat=0;

    printf("Enter num of process: ");
    scanf("%d", &n);

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    printf("Enter AT:");
    for (i=0; i<n; i++)
        scanf("%d", &at[i]);

    printf("Enter BT: ");
    for (i=0; i<n; i++) {
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    remain = n;

    for (time=0; i=0; remain!=0) {
        if (rt[i] < tq && rt[i] > 0) {
            time += rt[i];
            rt[i] = 0;
        }
    }
}
```

RT

16

17

14

14

0

0

0

23

Q/Ps-

Enter num of. Process: 5

Enter Time Quantum: 2

Enter AT:

0 1 2 3 4 5

Enter BT:

5 3 1 2 3

RRS Scheduling:

PID	AT	BT	CT	TAT	WT	RT
1	0	5	13	13	9	0
2	1	3	12	11	8	1
3	2	1	5	3	2	2
4	3	2	9	6	4	4
5	4	3	14	10	7	5

Avg TAT = 8.6000 ms.

Avg WT = 5.8000 ms.

Program -2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n"); int n, i;
printf("Enter number of processes: ");
scanf("%d", &n);

int at[n], bt[n], rt[n], ct[n], tat[n], wt[n], q[n];

printf("Enter Arrival Time:\n");
for(i = 0; i < n; i++)
scanf("%d", &at[i]);

printf("Enter Burst Time:\n");
for(i = 0; i < n; i++)
{ scanf("%d", &bt[i]);
rt[i] = bt[i];
}

printf("Enter Queue (1 = RR, 2 = FCFS):\n");
for(i = 0; i < n; i++)
scanf("%d", &q[i]);

int tq; printf("Enter Time
Quantum: "); scanf("%d", &tq);

int t = 0, completed = 0;
```



```
printf("\nGantt Chart:\n| ");
```

```
while(completed < n)
```

```
{ int found = 0;
```

```
for(i = 0; i < n; i++)
```

```
{ if(q[i] == 1 && rt[i] > 0 && at[i] <= t)
```

```
{ int exec = (rt[i] > tq) ? tq : rt[i];
```

```
for(int k = 0; k < exec; k++)
```

```
{ printf("P%d ", i+1);
```

```
t++; }
```

```
rt[i] -= exec;
```

```
if(rt[i] == 0)
```

```
{ ct[i] = t;
```

```
completed++; }
```

```
found = 1;
```

```
}
```

```
}
```

```
if(!found)
```

```
{ for(i = 0; i < n; i++)
```

```
{ if(q[i] == 2 && rt[i] > 0 && at[i] <= t)
```

```
{
```

```
while(rt[i] > 0)
```

```
{
```

```
// check if any Q1 arrives → break
```

```
int interrupt = 0;
```

```
for(int j = 0; j < n; j+
```

```
+)
```

```
{ if(q[j] == 1 && rt[j] > 0 && at[j] <= t)
```



```

        { interrupt = 1;
          break;
        }
      }
    if(interrupt) break;

    printf("P%d ", i+1);
    rt[i]--;
    ; t++; }

    if(rt[i] == 0)
    { ct[i] = t;
      completed++; }

    found = 1;
  }
}

// CPU idle
if(!found)
{
    printf("Idle "); t++;
}

printf("\n");

// Calculate TAT & WT for(i
= 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] -
bt[i]; }

```



```

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n"); for(i
= 0; i < n; i++)
{ printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

return 0;
}

```

Name: Nimay Anand Desai USN:1WA24CS189

Enter number of processes: 4

Enter Arrival Time:

0 0 0 10

Enter Burst Time:

4 3 8 5

Enter Queue (1 = RR, 2 = FCFS):

1 1 2 1

Enter Time Quantum: 2

Gantt Chart:

| P1 P1 P2 P2 P1 P1 P2 P3 P3 P3 P4 P4 P4 P4 P4 P3 P3 P3 |

Process	AT	BT	CT	TAT	WT

P1	0	4	6	6	2
P2	0	3	7	7	4
P3	0	8	20	20	12
P4	10	5	15	5	0

PS C:\Users\User\Documents\LAB_REPORT> █

24

Program-8

```

→ #include <stdio.h>
int main() {
    int n1, n2;
    int i;
    printf("Enter no. of processes: ");
    scanf("%d", &n1);
    printf("Enter priority: ");
    scanf("%d", &n2);

    int b1[20], b2[20];
    printf("Enter burst times: ");
    for (int i=0; i<n1; i++) {
        scanf("Q1", &i);
        scanf("%d", &b1[i]);
    }

    printf("Enter the burst times for queue 2: ");
    for (i=0; i<n2; i++) {
        printf("Q1 = P %d", i+1);
        scanf("%d", &b2[i+1]);
    }

    int time = 0;
    printf("Execution time: ");

    for (int i=0; i<n1; i++) {
        printf("t: %d to t = %d = Q1 = P = %d",
            time, time + b1[i], i+1);
    }

    for (i=0; i<n2; i++) {
        printf("t = %d to t = %d = Q2 = P = %d", time,
            time + b2[i], i+1);
        time = time + b2[i];
    }
}

```

25

O/p:-

Enter no. of
Enter no. of

Q1 — P1
Q1 — P2

Enter burst

Q2 → P

Q2 →

Execution

t = 0

t = 0

t = 3

t = 9

O/p:-

Enter no. of process in Q1 = 2

Enter no. of process in Q2 = 2

Q1 $\rightarrow$ P1 = 3

Q1 $\rightarrow$ P2 = 2

Enter burst time for Q2:

Q2 $\rightarrow$ P1 = 4

Q2 $\rightarrow$ P2 = 1

Execution Order:

t = 0

t = 3

Q1 $\rightarrow$ P1

t = 3

t = 3

Q1 $\rightarrow$ P2

t = 5

t = 9

Q2 $\rightarrow$ P1

t = 9

t = 10

Q2 $\rightarrow$ P2

Q

Program -3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms: (Any one)

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

RATE – MONOTONIC

CODE:

```
#include <stdio.h>
#include<math.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, i, j, t = 0, lcm = 1;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int burst[n], period[n], remaining[n];

    printf("Enter CPU burst times:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &burst[i]);

    printf("Enter time periods:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &period[i]);
    for(i = 0; i < n; i++)
    { int max = (lcm > period[i]) ? lcm : period[i];
        while(1)
        { if(max % lcm == 0 && max % period[i] == 0)
            { lcm = max;
                break;
            } max+
            +;
        } }
}
```



```

printf("\nLCM=%d\n", lcm);

printf("\nRate Monotonic Scheduling:\n");
printf("PID\tBurst\tPeriod\n"); for(i = 0; i
< n; i++)
    printf("%d\t%d\t%d\n", i+1, burst[i], period[i]);

float util = 0; for(i
= 0; i < n; i++)
    util += (float)burst[i] / period[i];

float bound = n * (pow(2, (float)1/n) - 1);

printf("\n%f <= %f => %s\n", util, bound,
    (util <= bound) ? "true" : "false");
for(i = 0; i < n; i++)
    remaining[i] = 0;

printf("\nScheduling occurs for %d ms\n\n", lcm);

for(t = 0; t < lcm; t++)
{

    for(i = 0; i < n; i++)
    { if(t % period[i] == 0)
        remaining[i] = burst[i];
    }

    int min = -1; for(i =
0; i < n; i++)
    { if(remaining[i] > 0)
        { if(min == -1 || period[i] < period[min])
            min = i;
        } }
}

```



```

    if(min != -1)
    { printf("%dms onwards: Process %d running\n", t, min+1);
      remaining[min]--;
    }
    else
    { printf("%dms onwards: CPU is idle\n", t);
      } }
    return 0;
}

```

Result:

```

Enter CPU burst times:
20 35
Enter time periods:
50 100

LCM=100

Rate Monotonic Scheduling:
PID    Burst    Period
1       20       50
2       35       100

0.750000 <= 0.828427 => true

Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
5ms onwards: Process 1 running
6ms onwards: Process 1 running
7ms onwards: Process 1 running
8ms onwards: Process 1 running
9ms onwards: Process 1 running
10ms onwards: Process 1 running
11ms onwards: Process 1 running
12ms onwards: Process 1 running
13ms onwards: Process 1 running
14ms onwards: Process 1 running
15ms onwards: Process 1 running
16ms onwards: Process 1 running
17ms onwards: Process 1 running
18ms onwards: Process 1 running
19ms onwards: Process 1 running
20ms onwards: Process 2 running
21ms onwards: Process 2 running
22ms onwards: Process 2 running
23ms onwards: Process 2 running
24ms onwards: Process 2 running
25ms onwards: Process 2 running
26ms onwards: Process 2 running
27ms onwards: Process 2 running
28ms onwards: Process 2 running
29ms onwards: Process 2 running
30ms onwards: Process 2 running

```

```

31ms onwards: Process 2 running
32ms onwards: Process 2 running
33ms onwards: Process 2 running
34ms onwards: Process 2 running
35ms onwards: Process 2 running
36ms onwards: Process 2 running
37ms onwards: Process 2 running
38ms onwards: Process 2 running
39ms onwards: Process 2 running
40ms onwards: Process 2 running
41ms onwards: Process 2 running
42ms onwards: Process 2 running
43ms onwards: Process 2 running
44ms onwards: Process 2 running
45ms onwards: Process 2 running
46ms onwards: Process 2 running
47ms onwards: Process 2 running
48ms onwards: Process 2 running
49ms onwards: Process 2 running
50ms onwards: Process 2 running
51ms onwards: CPU is idle
52ms onwards: CPU is idle
53ms onwards: CPU is idle
54ms onwards: CPU is idle
55ms onwards: CPU is idle
56ms onwards: CPU is idle
57ms onwards: CPU is idle
58ms onwards: CPU is idle
59ms onwards: CPU is idle
60ms onwards: CPU is idle
61ms onwards: CPU is idle
62ms onwards: CPU is idle
63ms onwards: CPU is idle
64ms onwards: CPU is idle
65ms onwards: CPU is idle
66ms onwards: CPU is idle
67ms onwards: CPU is idle
68ms onwards: CPU is idle
69ms onwards: CPU is idle
70ms onwards: CPU is idle
71ms onwards: CPU is idle
72ms onwards: CPU is idle
73ms onwards: Process 2 running
74ms onwards: Process 2 running
75ms onwards: CPU is idle
76ms onwards: CPU is idle
77ms onwards: CPU is idle
78ms onwards: CPU is idle
79ms onwards: CPU is idle
80ms onwards: CPU is idle
81ms onwards: CPU is idle
82ms onwards: CPU is idle
83ms onwards: CPU is idle
84ms onwards: CPU is idle
85ms onwards: CPU is idle
86ms onwards: CPU is idle
87ms onwards: CPU is idle
88ms onwards: CPU is idle
89ms onwards: CPU is idle
90ms onwards: CPU is idle
91ms onwards: CPU is idle
92ms onwards: CPU is idle
93ms onwards: CPU is idle
94ms onwards: CPU is idle
95ms onwards: CPU is idle
96ms onwards: CPU is idle
97ms onwards: CPU is idle
98ms onwards: CPU is idle
99ms onwards: CPU is idle
100ms onwards: CPU is idle

```


26

Program - 8

Rate Monotonic Scheduling

```
#include <stdio.h>
int gcd (int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
}
```

```
return a;
}
```

```
int sum (int a, int b) {
    return (a / gcd(a, b) * b);
}
```

```
int main () {
    printf ("Enter no. of tasks : ");
    scanf ("%d", &n);
    int t[20], p[20], id[20];
    int i, j;
```

```
for (i = 0; i < n; i++) {
    printf ("Enter comp time (%d): ", i + 1);
    scanf ("%d", &t[i]);
    printf ("Enter period (%d): ", i + 1);
    scanf ("%d", &p[i]);
    id[i] = i;
}
```

for (i = 0;


```

for (i=0; i<n-1; i++) {
    for (j=i+1; j<n; j++) {
        if (p[idn[i]] < p[idn[j]])
            int t = idn[i];
            idn[i] = idn[j];
            idn[j] = t;
    }
    int hyperperiod = p[0];
    for (i=1; i<n; i++)
        hyperperiod = lcm(hyperperiod, p[i]);
    if (sum == -1) {
        for (i=0; i<n; i++) {
            int t = idn[i];
            printf("%d ", t);
        }
        return 0;
    }
}

```

O/p:-

<u>PID</u>	<u>Burst</u>	<u>Period</u>
1	20	50
2	35	100

Scheduling occurs for 100ms

0	P1
20	P2
50	P1
70	P2
75	CPU idle.

EARLIEST DEADLINE FIRST

CODE:

```
#include <stdio.h>
#include <limits.h>

int main() {
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, i, time, hyperperiod = 20;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    int C[n], D[n], T[n];
    int remaining[n];
    int next_arrival[n];
    int abs_deadline[n];

    for(i = 0; i < n; i++) {
        printf("Enter Capacity, Deadline, Period for Task %d: ", i + 1);
        scanf("%d %d %d", &C[i], &D[i], &T[i]);

        remaining[i] = 0;
        next_arrival[i] = 0;
        abs_deadline[i] = 0;
    }

    printf("\n--- EDF Scheduling (Periodic Tasks) ---\n");

    for(time = 0; time < hyperperiod; time++) {
        for(i = 0; i < n; i++) { if(time
            == next_arrival[i]) {
                remaining[i] = C[i];
                abs_deadline[i] = time + D[i];
                next_arrival[i] += T[i];
            }
        }
    }
}
```



```

int selected = -1; int
min_deadline = INT_MAX;

for(i = 0; i < n; i++) {
    if(remaining[i] > 0 && abs_deadline[i] < min_deadline) {
        min_deadline = abs_deadline[i];
        selected = i;
    }
}

if(selected == -1) {
    printf("%d ms : CPU is idle.\n", time);
} else { printf("%d ms : Task %d is running.\n", time,
    selected + 1); remaining[selected]--;
} }

return 0;
}

```

Result:

Name: Nimay Anand Desai
USN: 1WA24CS189

Enter number of tasks:
3

Enter Capacity, Deadline, Period for Task 1: 3 7 20
Enter Capacity, Deadline, Period for Task 2: 2 4 5
Enter Capacity, Deadline, Period for Task 3: 2 8 10

EDF Scheduling (Periodic Tasks) ---

0 ms : Task 2 is running.
1 ms : Task 2 is running.
2 ms : Task 1 is running.
3 ms : Task 1 is running.
4 ms : Task 3 is running.
5 ms : Task 3 is running.
6 ms : Task 3 is running.
7 ms : Task 2 is running.
8 ms : Task 2 is running.
9 ms : CPU is idle.
10 ms : Task 2 is running.
11 ms : Task 2 is running.
12 ms : Task 3 is running.
13 ms : Task 3 is running.
14 ms : CPU is idle.
15 ms : Task 1 is running.
16 ms : Task 2 is running.
17 ms : CPU is idle.
18 ms : CPU is idle.
19 ms : CPU is idle.

28

Program - 10

Earliest deadline first:-

```
#include <stdio.h>
#define INF 100000
```

```
int main() {
    int n;
    printf("Enter Process: ");
    scanf("%d", &n);
    int ar[50], br[50], dead[50];
    int rem[50];
    int comp = 0;
    int time = 0;
```

```
for (int i = 0; i < n; i++) {
    printf("Enter AT, BT, deadline for P%d: ", i+1);
    scanf("%d %d %d", &ar[i], &br[i], &dead[i]);
    rem[i] = br[i];
}
```

```
printf("Execution timeline: ");
```

```
while (comp < n) {
    int min_deadline = INF;
    int sl = -1;
```

```
for (int i = 0; i < n; i++) {
    if (ar[i] <= time & rem[i] > 0) {
        if (dead[i] < min) {
            min = dead[i];
            sl = i;
        }
    }
}
```

18

3

3

0.18

En

En

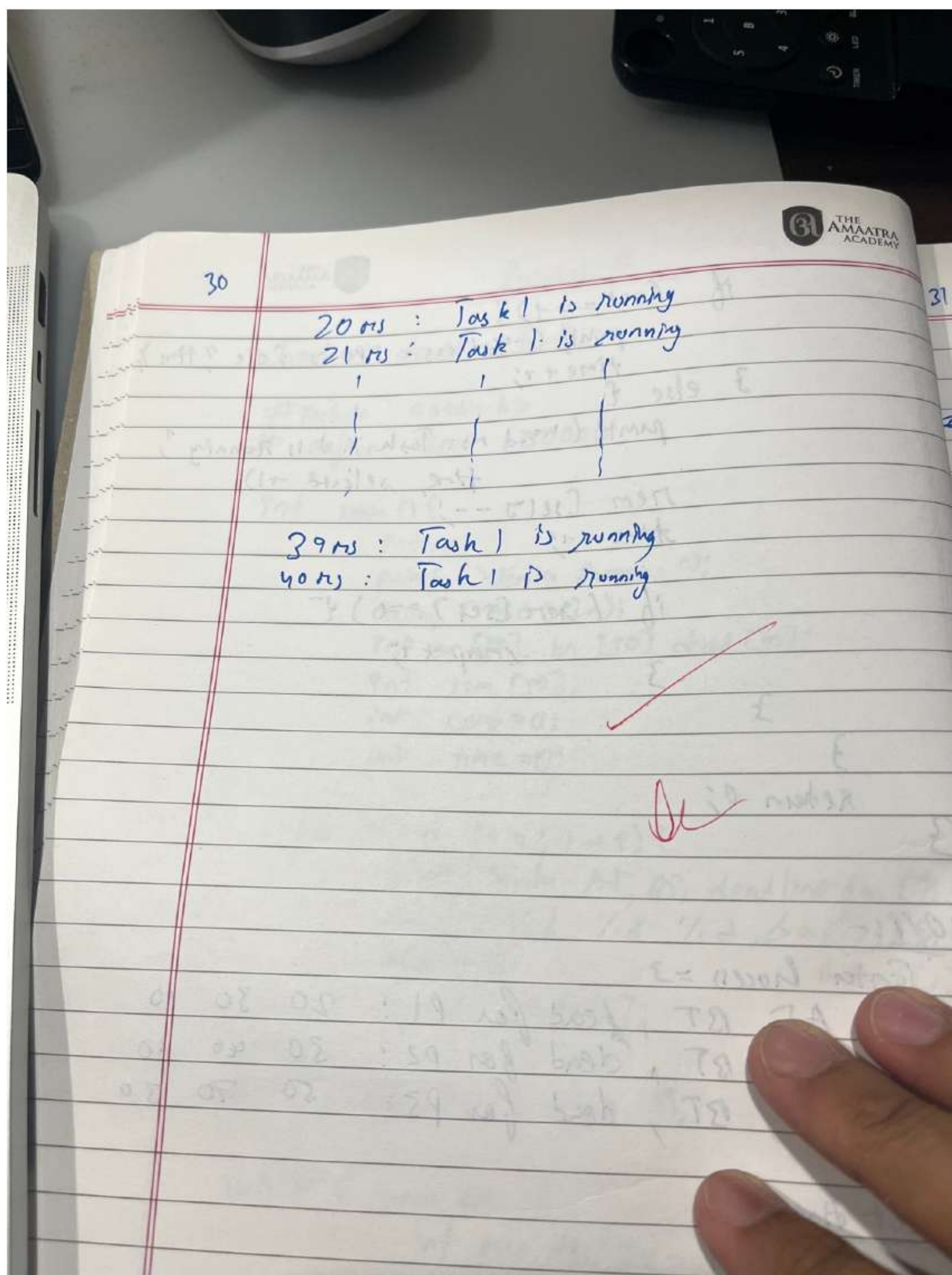
En

1

$\frac{1}{2} \log \left(\frac{1}{2} \right)$
 $\frac{1}{2} \log \left(\frac{1}{2} \right)$
 $\frac{1}{2} \log \left(\frac{1}{2} \right)$

Enter AT, BT, dead for P1: 20 30 10
Enter AT, BT, dead for P2: 30 40 60
Enter AT, BT, dead for P3: 50 70 30

$$\frac{>}{r}$$



PROPORTIONAL SHARE SCHEDULING

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    char
name[5]; int
tickets; }
Process;

int main() {
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n");
    int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes:
"); scanf("%d", &n); Process p[n];
    srand(time(NULL));

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i +
1); printf("Tickets: ");
        scanf("%d", &p[i].tickets);
        total_tickets += p[i].tickets;
        total_T += p[i].tickets;
    }
    printf("\n--- Proportional Share Scheduling ---
\n"); printf("Enter the Time Period for scheduling:
"); int m; scanf("%d", &m);

    for (int i = 0; i < m; i++) { int winning_ticket =
        rand() % total_tickets + 1; int
        accumulated_tickets = 0; int winner_index;
```



```

for (int j = 0; j < n; j++) {
    accumulated_tickets += p[j].tickets; if
    (winning_ticket <= accumulated_tickets) {
        winner_index = j;
        break;
    } }

printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
    p[winner_index].name);
}

for (int i = 0; i < n; i++) {
    printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
        ((p[i].tickets / total_T) * 100));
}

return 0;
}

```

Result:


```
Name: Nimay Anand Desai
USN: 1WA24CS189

Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5

Tickets picked: 35, Winner: P3
Tickets picked: 52, Winner: P3
Tickets picked: 9, Winner: P1
Tickets picked: 15, Winner: P2
Tickets picked: 7, Winner: P1

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
PS C:\Users\User\Documents\LAB_REPORT> █
```


Proportional Share Scheduling :-

```

→ #include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

```

typedef struct {
    char name[5]; int tic;
}

```

```

Process;

```

```

int main () {
    int n, total-tic=0; float total-t=0.0;

```

```

    printf("Enter process: ");

```

```

    scanf("%d", &n);

```

```

    process p[n];

```

```

    srand [time (now)];

```

```

    for (int i=0; i<n; i++) {

```

```

        printf("Process %d", i+1);

```

```

        sprintf(p[i].name, "p%d", i+1);

```

```

        printf("Tickets: ");

```

```

        scanf("%d", &p[i].tickets);

```

```

        total-tic += p[i].tic;

```

```

        total-T += p[i].tickets;
    }

```

```

    printf("Proportional share scheduling: ");

```

```

    printf("Enter time period: ");

```

```

    int m;

```

```

    scanf("%d", &m);

```



```

for (int i = 0; i < n; i++) {
    int win = rand() % (total - tick + 1);
    int accumulated - tick = 0;
    int winner;

    for (int j = 0; j < n; j++) {
        acc += p[i] - tickets;
        if (win <= acc - tick) {
            winner - index = j;
            break;
        }
    }

    printf("Tickets p[del] : %d, winner = %d,
           winner - tick, P[winner - index].name);
}

for (int i = 0; i < n; i++) {
    printf("The process %s gets %.02f
           %s process time, p[i].name,
           (1 - p[i].tickets / total - 1) * 100);
}

return 0;
}

```


34

Output:-

Enter no. of process = 3

P1

Tickets = 10

P2

Tickets = 20

P3

Tickets = 30

Proportional share scheduling

Enter the Time Period in $\sim$ 5

Tickets Picked = 20, Winner = P2

Tickets Picked = 55, Winner = P3

Tickets Picked = 67, Winner = P3

Tickets Picked = 80, Winner: P2

The process P1 gets 16.67% process time.

The process P2 gets 53.33% process time.

The process P3 gets 50% of process time.

Program -4

Question:

Write a C program to simulate: (Any one)

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

PRODUCER – CONSUMER

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int
buffer[BUFFER_SIZE];
int top = -1; int item = 0;

sem_t empty, full, mutex;

void produce() {
    sem_wait(&empty);
    sem_wait(&mutex);

    item++; top++;
    buffer[top] = item;

    printf("Producer has produced: Item %d\n", item);
    sem_post(&mutex); sem_post(&full);
}

void consume() {
    sem_wait(&full);
    sem_wait(&mutex);

    int consumed = buffer[top];
```

```

printf("Consumer has consumed: Item %d\n", consumed);

top--;

sem_post(&mutex);
sem_post(&empty);
}

int main() {
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
int choice;

sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);
sem_init(&mutex, 0, 1);

while (1) {
printf("\nEnter 1. Producer 2. Consumer 3.
Exit\n"); printf("Enter your choice: "); scanf("%d",
&choice);

switch (choice)
{ case 1:
if (top == BUFFER_SIZE - 1)
{ printf("Buffer is full!\n");
} else
{ produce();
} break;

case 2:
if (top == -1) {
printf("Buffer is empty!\n");
} else
{ consume();
} break;

case 3:

```



```

    printf("Exiting...\n");
    exit(0);

default:
    printf("Invalid choice!\n");
} }

return 0;
}
Result:

```

USN: 1WA24CS189

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 1

Producer has produced: Item 1

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 1

Producer has produced: Item 2

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 2

Consumer has consumed: Item 2

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 2

Consumer has consumed: Item 1

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 2

Buffer is empty!

Enter 1. Producer 2. Consumer 3. Exit

Enter your choice: 3

Exiting...

Producer-Consumer:-

```
#include <stdio.h>
#include <stdlib.h>
```

```
int mutex = 1;
int full = 0;
int empty = 5;
int n = 0;
int buffer[5];
int i = 0;
int out = 0;
```

```
int wait (int s) {
    return [--s];
}
```

```
int signal (int s) {
    return [++s];
}
```

```
void producer () {
    if (mutex == 1 && empty != 0) {
        mutex = wait (mutex);
        full = signal (full);
        empty = wait (empty);
```

```
        n++;
```

```
        buffer [in] = n;
```

```
        printf ("Producer (consumer. exit 1) item %d  
at slot %d", n, in);
```

```
        while (1) {
```

```
            printf ("Enter
```



```

in = (in+1) % 5;
mutex = signal (mutex);
} else {
    printf ("Buffer ");
}
}

void consumer () {
    if (mutex == 1)
        kill = wait (&kill);
    empty = signal (empty);
    int item = buffer (out);
    printf ("Consumer consumed %d from slot\n", item);
    out = (out+1) % 5;
    mutex = signal (mutex);
} else {
    printf ("Buffer empty");
}
}

```

```

int main () {
    int n;
    printf ("Producers Consumer Exit\n");
    while (1) {
        printf ("Enter your choice: ");
        scanf ("%d", &n);
    }
}

```



```
switch (n) {
```

```
    case 1:
```

```
        producer();  
        break;
```

```
    case 2:
```

```
        consumer();  
        break;
```

```
    case 3:
```

```
        printf("Exit");  
        exit(0);  
        break;
```

```
    default:
```

```
        printf("Invalid");
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```

O/p:-

1. Producer
2. Consumer
3. Exit

Enter your choice = 1

Producer produces item 1 at slot 0

Enter your choice = 2

Consumer consumes item 1 from slot 0.

Producer produces item 2 at slot 1.

Enter your choice 2.

Consumer consumes item 2 from slot 2.

Enter your choice 3

Exiting program.

DINING – PHILOSOPHER’S CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void* philosopher(void* num) {
    int id = *(int*)num; int
    left = id; int right = (id
    + 1) % N;

    for (int i = 0; i < 1; i++) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]); printf("Philosopher
            %d picked left fork %d\n", id, left);

            pthread_mutex_lock(&forks[right]); printf("Philosopher
            %d picked right fork %d\n", id, right);
        } else { pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d\n", id,
            right);

            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked left fork %d\n", id, left);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(2);
    }
}
```



```

        pthread_mutex_unlock(&forks[left]); pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d\n", id, left, right);
    }

    return NULL;
}

int main() {
    printf("Name: Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int ids[N];

    for (int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i; }

    for (int i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }

    for (int i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}

```

Result:

Result:

Name: Nimay Anand Desai

USN: 1WA24CS189

```
Philosopher 0 is thinking.  
Philosopher 1 is thinking.  
Philosopher 2 is thinking.  
Philosopher 3 is thinking.  
Philosopher 0 picked left fork 0  
Philosopher 1 picked right fork 2  
Philosopher 1 picked left fork 1  
Philosopher 1 is eating.  
Philosopher 3 picked right fork 4  
Philosopher 3 picked left fork 3  
Philosopher 3 is eating.  
Philosopher 1 put down forks 1 and 2  
Philosopher 0 picked right fork 1  
Philosopher 0 is eating.  
Philosopher 2 picked left fork 2  
Philosopher 3 put down forks 3 and 4  
Philosopher 2 picked right fork 3  
Philosopher 2 is eating.  
Philosopher 0 put down forks 0 and 1  
Philosopher 2 put down forks 2 and 3
```


40

```
→ #include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
#define N 5
```

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_t philosopher[N];
```

```
void *philosopher(void *num) {
    int id = *(int *) num;
    pthread_t tid = id;
    int right = (id + 1) % N;
```

```
    while (1) {
        printf("Philosopher %d\n", id);
        sleep(1);
```

```
        if (id % 2 == 0) {
            pthread_mutex_lock(&mutex);
            printf("Philosopher %d left fork\n", id);
```

```
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up %d\n",
                id, right);
```

```
        }
    }
}
```



```

pthread_mutex_lock(&forks(right));
printf("philosopher %d picked up right fork %d, id,
right);
pthread_mutex_lock(&forks(left));
printf("philosopher %d picked up left fork %d,
id, left);
}
printf("philosopher %d is eating ", id);
sleep(2);

pthread_mutex_unlock(&forks(left));
pthread_mutex_unlock(&forks(right));

printf("philosopher %d put down forks %d & %d
id, left, right);
}
return NULL;
}

int main() {
int i;
int ids[N];

for (i=0; i<N; i++) {
pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
}
for (i=0; i<N; i++) {
pthread_create(&philosophers[i], NULL, philosopher,
&ids[i]);
}
for (i=0; i<N; i++) {
pthread_join(philosophers[i], NULL);
}
}

```


42

```

for (i=0; i<N; i++) {
    pthread_mutex_lock(&forks[i]);
    return 0;
}

```

O/P

Philosopher 0 is thinking
 Philosopher 2 is thinking.
 Philosopher 4 is thinking
 Philosopher 3 is Thinking

De

Berkeley's A

→ # include
int

Program -5

Question:

Write a C program to simulate: (Any one)

- a) Bankers' algorithm for the purpose of deadlock avoidance.
- b) Deadlock Detection

BANKER'S ALGORITHM

CODE:

```
#include <stdio.h>

int main()
{
printf("Name: Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    { for(j = 0; j < m; j++)
        { scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter Maximum Demand Matrix:\n");
    for(i = 0; i < n; i++)
    { for(j = 0; j < m; j++)
        { scanf("%d", &max[i][j]);
        } }

    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
```



```

{ scanf("%d", &avail[i]);
}

// Calculate Need Matrix
for(i = 0; i < n; i++)
{ for(j = 0; j < m; j++)
    { need[i][j] = max[i][j] - alloc[i][j];
    } }

int finish[n], safeSeq[n];
int work[m];

for(i = 0; i < n; i++)
    finish[i] = 0;

for(i = 0; i < m; i++)
    work[i] = avail[i];

int count = 0;

while(count < n)
{ int found = 0;

    for(i = 0; i < n; i++)
    { if(finish[i] == 0)
        { int possible = 1;

            for(j = 0; j < m; j++)
            { if(need[i][j] > work[j])
                { possible = 0;
                break;
                }
            }

            if(possible)
            { for(k = 0; k < m; k++)

```

```

        { work[k] += alloc[i][k];
        }

        safeSeq[count] = i; count+
        +;
        finish[i] = 1;
        found = 1;
    }
} }

if(found == 0)
{ printf("System is not in a safe state.\n");
  return 0;
} }

printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{ printf("P%d", safeSeq[i]);

    if(i != n - 1)
        printf(" -> ");
}

printf("\n");

return 0;
}
Result:

```


Name: Nimay Anand Desai

USN: 1WA24CS189

Enter number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

System is in a safe state.

Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Program-14

Banker's Algorithm:-

```

→ #include <stdio.h>
int main () {
    int n, m, i, j, k;
    printf ("Banker's algorithm.");
    printf ("Enter no. of process: ");
    scanf ("%d", &n);

    printf ("Enter no. of resources: ");
    scanf ("%d", &m);

    int alloc [n][m], max [n][m], need [n][m];
    int avail [m];

    printf ("Enter alloc matrix:");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf ("%d", &alloc [i][j]);
        }
    }

    printf ("Enter maximum demand Matrix:");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf ("%d", &max [i][j]);
        }
    }

    printf ("Enter available resources:");

```



```

for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

```

```

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j] = min[i][j] - avail[j];
    }
}

```

```

int finish[n], safe_req(n), work[m];

```

```

for (i=0; i<n; i++)
    finish[i] = 0;

```

```

for (i=0; i<m; i++)
    work[i] = avail[i];

```

```

int count = 0;

```

```

while (count < n) {

```

```

    int found = 0;

```

```

    for (i=0; i<n; i++) {
        if (finish[i] == 0) {

```

```

            int pass = 1;

```

```

            for (j=0; j<m; j++) {

```

```

                if (need[i][j] > work[j])

```

```

                    pass = 0;

```

```

            }

```

```

        }

```


95

if (possible) {
 for (k=0; k<m; k++)
 work(k) += alloc r[i][k];
 safeseg [count++] = i;
 finish[i] = 1;
 bound = 1;
 }
 }
 if (bound == 0)
 break;
 }
 if (count == n) {
 printf("system in safe state");
 printf("safe seq is: ");
 for (i=0; i<n; i++) {
 printf("%d", safeseg[i]);
 if (i != n-1)
 printf(" ");
 }
 printf("\n");
 }
 else if
 printf("System is NOT in a safe state");
 }
 return 0;
 }

O/P:-

Enter no. of process = 5
 Enter no. of resources = 3
 Enter allocation matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter Max Demand Matrix:

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

Enter avail resources:

3	3	2
---	---	---

System is in safe state.

Safe sequence: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$.

Safety

1. Let w
 initial

2. F

3.

4.

P_0

P_1

P_2

P_3

P_4

DEADLOCK DETECTION

CODE:

```
#include <stdio.h>

int main()
{
printf("Name:Nimay Anand Desai\nUSN:1WA24CS189\n\n");
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], req[n][m]; int
    avail[m], finish[n], safe[n];

    printf("Enter allocation matrix:\n");
    for(i = 0; i < n; i++)
    { printf("Process %d: ", i);
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    }

    printf("Enter request matrix:\n");
    for(i = 0; i < n; i++)
    { printf("Process %d: ", i);
        for(j = 0; j < m; j++)
            scanf("%d", &req[i]
[j]); }

    printf("Enter available resources: ");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);
```



```

for(i = 0; i < n; i++)
    finish[i] = 0;

int count = 0;

while(count < n)
{ int found = 0;

    for(i = 0; i < n; i++)
    { if(finish[i] == 0)
        { int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(req[i][j] > avail[j])
                { possible = 0;
                    break;
                }
            }

            if(possible)
            { for(k = 0; k < m; k++)
                avail[k] += alloc[i][k];

                safe[count] = i;
                count++;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
        break;
}

if(count == n)

```

```

{ printf("\nSystem is in safe state.\n");
  printf("Safe Sequence is: ");

  for(i = 0; i < n; i++)
    printf("P%d ",
safe[i]); }
else
{ printf("\nDeadlock detected.\n");
}

return 0;
}

```

```

printf("\nEnter Allocation\n");
for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n], work[m];

for (i=0; i<n; i++)
    finish[i] = 0;

for (i=0; i<m; i++)
    work[i] = avail[i];

int count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            int poss = 1;
            for (j=0; j<m; j++) {
                if (need[i][j] > work[j])
                    poss = 0;
            }
            if (poss) {
                for (j=0; j<m; j++)
                    work[j] += alloc[i][j];
                finish[i] = 1;
                count++;
            }
        }
    }
}

```



```

Deadlock detection also
#include <stdio.h>

int main() {
    int n, m, i, j, t;
    printf("Deadlock detection");
    printf("Enter no. of process: ");
    scanf("%d", &n);

    printf("Enter the no. of resources: ");
    scanf("%d", &m);

    int alloc[n][m];
    int req[n][m];
    int avail[n][m];

    printf("Enter allocation matrix:");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter the req matrix:");
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &req[i][j]);
        }
    }
}

```

```
for (i=0; i<m; i++)
```

```
scanf("%d", &avail[i]);
```

```
{
```

```
int work[m];
```

```
int finish[n];
```

```
int select[n];
```

```
for (i=0; i<m; i++)
```

```
work[i] = avail[i];
```

```
for (i=0; i<n; i++)
```

```
int zero = 1;
```

```
for (j=0; j<m; j++)
```

```
if (alloc[i][j] != 0)
```

```
zero = 0;
```

```
break;
```

```
}
```

```
if (zero)
```

```
finish[i] = 1;
```

```
else
```

```
finish[i] = 0;
```

```
{
```

```
int count = 0;
```

```
while (count < n)
```

```
int found = 0;
```

```
for (i=0; i<n; i++)
```

```
if (finish[i] == 0)
```

```
int poss = 1;
```



```

for (j=0; j < m; j++)
    if (req[i][j] > work[j])
        poss = 0;
        break;
}

```

```

if (poss)
    for (k=0; k < m; k++)
        work[k] += alloc[i][k];
    safe_seq[count++] = i;
    finish[i] = 1;
    found = 1;
}
}
}

```

```

if (found == 0)
    break;
}
}

```

```

for (i=0; i < n; i++)
    if (finish[i] == 0)
        deadlock = 1;
        break;
}
}

```

```

if (deadlock == 0)
    printf("sys in safe seq : ");
    printf("safe seq is: ");
    for (i=0; i < count; i++)
        printf("pid : %d, safe seq : %d\n", i, safe_seq[i]);
    printf("\n");
}
else

```

```

for (i=0; i<n; i++) {
    if (finish[i] == 0)
        printf("P%d ", i);
}
return 0;
}

```

O/P:

Enter the no. of process : 5

Enter the no. of resource : 3

Enter alloc matrix:

0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter the request matrix:

0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter the available resource: 0 0 0
in safe state

Program -6

Question:

Write a C program to simulate the following contiguous memory allocation techniques. (Any

one) a) Worst-fit

b) Best-fit

c) First-fit

CODE:

```
#include <stdio.h>
```

```
void firstFit(int blocks[], int m, int processes[], int n) {
```

```
    int alloc[n], used[m];
```

```
    for (int i = 0; i < m; i+
```

```
+) used[i] = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
        alloc[i] = -1;
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            if (!used[j] && blocks[j] >= processes[i]) {
```

```
                alloc[i] = j;
```

```
                used[j] = 1;
```

```
                break;
```

```
            }
```

```
    printf("\n--- First Fit ---\n");
```

```
    printf("Process No.\tProcess Size\tBlock No.\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d\t%d\t", i + 1, processes[i]);
```

```
        if (alloc[i] != -1)
```

```
            printf("%d\n", alloc[i] + 1);
```

```
        else
```

```
            printf("Not Allocated\n");
```

```
    } }
```

```
void bestFit(int blocks[], int m, int processes[], int n) {
```

```

int alloc[n], used[m];
for (int i = 0; i < m; i++)
    used[i] = 0;
for (int i = 0; i < n; i++)
    alloc[i] = -1;

for (int i = 0; i < n; i++) {
    int best = -1;
    for (int j = 0; j < m; j++)
        if (!used[j] && blocks[j] >= processes[i])
            if (best == -1 || blocks[j] < blocks[best])
                best = j;
    if (best != -1)
        { alloc[i] = best;
          used[best] = 1;
        }
}

printf("\n--- Best Fit ---\n");
printf("Process No. \tProcess Size\tBlock No.\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else printf("Not Allocated\n");
}

}

void worstFit(int blocks[], int m, int processes[], int n) {
    int alloc[n], used[m];
    for (int i = 0; i < m; i++)
        used[i] = 0;
    for (int i = 0; i < n; i++)
        alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int worst = -1;
        for (int j = 0; j < m; j++)

```



```

        if (!used[j] && blocks[j] >= processes[i])
            if (worst == -1 || blocks[j] > blocks[worst])
                worst = j;
    if (worst != -1)
        { alloc[i] = worst;
          used[worst] = 1;
        } }

printf("\n--- Worst Fit ---\n"); printf("Process No.
\tProcess Size\tBlock No.\n"); for (int i = 0; i <
n; i++) {
    printf("%d\t%d\t", i + 1, processes[i]);
    if (alloc[i] != -1)
        printf("%d\n", alloc[i] + 1);
    else printf("Not
        Allocated\n");
}
}
int main() {

    printf("Name: Nimay Anand Desai\nUSN:1WA24CS189\n\n");

    int m, n;

    printf("Enter number of memory blocks:
    "); scanf("%d", &m); int blocks[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes:
    "); scanf("%d", &n); int
    processes[n];
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &processes[i]);

```

```

firstFit(blocks, m, processes, n);
bestFit(blocks, m, processes, n);
worstFit(blocks, m, processes, n);

return 0;
}

```

Name: Nimay Anand Desai
USN: 1WA24CS189

Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
2122 417 112 426

--- First Fit ---

Process No.	Process Size	Block No.
1	2122	Not Allocated
2	417	2
3	112	3
4	426	5

--- Best Fit ---

Process No.	Process Size	Block No.
1	2122	Not Allocated
2	417	2
3	112	3
4	426	5

--- Worst Fit ---

Process No.	Process Size	Block No.
1	2122	Not Allocated
2	417	5
3	112	2
4	426	Not Allocated

==== Execution Successful =====

300	512
200	
700	812
400	212
F	

600	512
300	212
200	
700	526
400	312
B	

600	512
300	
200	
700	212
400	312
W	

Code:-

```
#include <stdio.h>
void firstfit(int blockSize[], int m, int processSize[], int n)
```

```
{
    int allocation[n];
```

```
    for (int i=0; i<n; i++)
        allocation[i] = -1;
```

```
    for (int i=0; i<n; i++)
    {
```

```
        for (int j=0; j<m; j++)
        {
```

```
            if (blockSize[j] >= processSize[i])
```

```
            {
                allocation[i] = j;
```

```
                blockSize[j] -= processSize[i];
                break;
            }
```

```
        }
```

```
    }
```

```
printf("\n -- First fit -- \n");
```

```
printf("Process No. \t Process Size \t Block No. \n");
```

```
for (int i=0; i<n; i++)
{
```

```
}
```


56

```

printf("%d\t\t%d\t\t", i+1, processSize[i]);
if (allocation[i] != -1)
    printf("%d\n", allocation[i] - 1);
else
    printf("Not Allocated\n");
}
}

```

```

void bestFit(int blockSize[], int m, int processSize[], int n)
{

```

```

    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++)
    {

```

```

        int bestIdx = -1;
        for (int j = 0; j < m; j++)
        {

```

```

            if (blockSize[j] >= processSize[i])
            {

```

```

                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }

```

```

    if (bestIdx != -1)
    {

```

```

        allocation[i] = bestIdx;
        blockSize[bestIdx] -= processSize[i];
    }
}

```

```

printf("\n --- Best Fit --- \n");
printf("Process No. | Process Size | Block no. \n");
for (int i = 0; i < n; i++)
{

```


new Size [i];

57



```
printf ("%d \t\t %d \t\t", i+1, processize [i]);
if (allocation [i] != -1)
    printf ("%d \n", allocation [i] + 1);
else
    printf ("Not Allocated \n");
```

}

```
void wastFit (int Blocksize [], int m, int processize [], int n)
```

```
{
    int allocation [n];
    for (int i=0; i<n; i++)
        allocation [i] = -1;
    for (int i=0; i<n; i++)
```

```
{
    int wastIdx = -1;
    for (int j=0; j<m; j++)
```

```
{
    if (blocksize [j] <= processize [i])
```

```
{
    if (wastIdx == -1 || blocksize [j] < blocksize [wastIdx])
        wastIdx = j;
    }
```

}

```
if (wastIdx != -1) {
```

```
    allocation [i] = wastIdx;
    blocksize [wastIdx] -= processize [i];
}
```

}

```
printf ("\n --- Wast Fit --- \n");
```

```
printf ("Process No. \t Process size \t Block No. \n");
```

```
for (int i=0; i<n; i++) {
```

```
    printf ("%d \t\t %d \t\t", i+1, processize [i]);
```

```
    if (allocation [i] != -1)
        printf ("%d \n", allocation [i] + 1);
```

```
    else
        printf ("Not Allocated \n");
}
```


58

```

int main ()
{

```

```

    int m, n;
    printf ("Enter no. of memory blocks: ");
    scanf ("%d", &m);
    int blockSize [m], block1 [m], block2 [m], block3 [m];
    printf ("Enter sizes of %d memory blocks: \n", m);
    for (int i=0; i<m; i++)
    {

```

```

        scanf ("%d", &blockSize [i]);
        block1 [i] = blockSize [i];
        block2 [i] = blockSize [i];
        block3 [i] = blockSize [i];
    }

```

```

    printf ("Enter the no. of processes: ");
    scanf ("%d", &n);
    int processSize [n];
    printf ("Enter sizes of %d processes: \n", n);
    for (int i=0; i<n; i++)
        scanf ("%d", &processSize [i]);
    firstFit (block1, m, processSize, n);
    bestFit (block2, m, processSize, n);
    worstFit (block3, m, processSize, n);
    return 0;
}

```

Least Recent

Replace

	7	0	1
P ₁			
P ₃			
P ₂		0	
P ₄	7	7	
	x	x	

1	
2	
1	
0	
3	

L

5

P₃
P₂
P₁

Program -7

Question:

Write a C program to simulate page replacement algorithms. (Any one)

- a) FIFO
- b) LRU
- c) Optimal

FIFO

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Name:Nimay Anand Desai\nUSN: 1WA24CS189\n\n");
```

```
    int pages[20], frames[10]; int n, f,  
    i, j, k = 0, fault = 0, found;
```

```
    printf("Enter number of pages: ");  
    scanf("%d", &n);
```

```
    printf("Enter page reference string: ");  
    for(i = 0; i < n; i++) {  
        scanf("%d", &pages[i]);  
    }
```

```
    printf("Enter number of frames: ");  
    scanf("%d", &f);
```

```
    for(i = 0; i < f; i++) {  
        frames[i] = -1;  
    }
```

```
    for(i = 0; i < n; i++) {
```

```
        found = 0;
```

```
        for(j = 0; j < f; j++) {
```

```

        if(frames[j] == pages[i]) {
            found = 1;
            break;
        }

    if(found == 0)
        { frames[k] = pages[i];
          k = (k + 1) % f;
          fault+
+; }

    printf("\nPage %d -> ", pages[i]);

    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%d ", frames[j]);
        else
            printf("- ");
    }

    printf("\n\nTotal Page Faults = %d\n", fault);

    return 0;
}
Result:

```


Result

Name: Nimay Anand Desai

USN: 1WA24CS189

Enter number of pages: 12

Enter page reference string: 7 0 1 2 0 3 0 4 2 3 0 3

Enter number of frames: 2

Page 7 -> 7 -

Page 0 -> 7 0

Page 1 -> 1 0

Page 2 -> 1 2

Page 0 -> 0 2

Page 3 -> 3 2

Page 0 -> 0 3

Page 4 -> 4 3

Page 2 -> 4 2

Page 3 -> 3 2

Page 0 -> 3 0

Page 3 -> 3 0

Total Page Faults = 10

===== Execution Successful =====

60

FIPO:-

```
→ #include <stdio.h>
int main () {
```

```
    int pg, fr;
    int n, l, i, j;
    int ppf = 0;
    int next = 0;
    int found;
```

```
    printf("Enter no. of pg: ");
    scanf("%d", &n);
```

```
    printf("Enter no. of ref string: ");
    scanf("%d", &l);
```

```
    printf("Enter no. of frames: ");
    scanf("%d", &fr);
```

```
    for (i=0; i<fr; i++) {
        fr[i] = -1;
    }
```

```
    printf("FIPO:");
    for (i=0; i<n; i++) {
        found = 0;
```

```
        for (j=0; j<fr; j++) {
            if (frames[j] == page[i]) {
                found = 1;
                break;
            }
        }
```

```
    }
```

};

O/P:-

```
Enter
Enter
7
Enter
```



```

if (bound == 0) {
    frames[next] = page[i];
    next = (next + 1) % f;
    pf++;
}

printf("Page %d ", page[i]);
for (j = 0; j < f; j++) {
    if (frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf(" ");
}
printf("\n");
}
printf("Total no. of faults %d, pf);

return 0;
};

```

O/p:-

Enter no. of pages: 12

Enter page ref string:

7 0 1 2 0 3 0 4 2 3 0 3

Enter no. of frames: 3

62

$P7 \rightarrow [7]$
 $P0 \rightarrow [7 \ 0]$
 $P1 \rightarrow [7 \ 0 \ 1]$
 $P2 \rightarrow [2 \ 0 \ 1]$
 $P0 \rightarrow [2 \ 0 \ 1]$
 $P3 \rightarrow [2 \ 3 \ 1]$
 $P0 \rightarrow [2 \ 3 \ 0]$
 $P4 \rightarrow [4 \ 3 \ 0]$
 $P2 \rightarrow [4 \ 2 \ 0]$
 $P3 \rightarrow [4 \ 2 \ 3]$
 $P0 \rightarrow [0 \ 2 \ 3]$
 $P3 \rightarrow [0 \ 2 \ 3]$

Total page faults = 10

LRO:-

Pre
Pnt

LRU

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Name:Nimay Anand Desai\nUSN: 1WA24CS189\n\n");
```

```
    int pages[20], frames[10], time[10]; int n,  
    f, i, j, pos, fault = 0, count = 0, found;
```

```
    printf("Enter number of pages: ");  
    scanf("%d", &n);
```

```
    printf("Enter page reference string: ");  
    for(i = 0; i < n; i++) {  
        scanf("%d", &pages[i]);  
    }
```

```
    printf("Enter number of frames: ");  
    scanf("%d", &f);
```

```
    for(i = 0; i < f; i++) {  
        frames[i] = -1;  
    }
```

```
    for(i = 0; i < n; i++) {
```

```
        found = 0;
```

```
        for(j = 0; j < f; j++) {  
            if(frames[j] == pages[i]) {  
                count++;  
                time[j] =  
                count; found =  
                1; break;
```

```

    } }

if(found == 0) {

    pos = 0;

    for(j = 1; j < f; j++) {
        if(time[j] < time[pos]) {
            pos = j;
        } }

    frames[pos] =
pages[i]; count++;
time[pos] = count; fault+
+; }

printf("\nPage %d -> ", pages[i]);

for(j = 0; j < f; j++) {
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else printf("-
");
}
}

printf("\n\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Result:

Result

Name: Nimay Anand Desai

USN: 1WA24CS189

Enter number of pages: 12

Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5

Enter number of frames: 3

Page 1 -> 1 - -

Page 2 -> 1 2 -

Page 3 -> 1 2 3

Page 4 -> 4 2 3

Page 1 -> 4 1 3

Page 2 -> 4 1 2

Page 5 -> 5 1 2

Page 1 -> 5 1 2

Page 2 -> 5 1 2

Page 3 -> 5 3 2

Page 4 -> 5 3 4

Page 5 -> 5 3 4

Total Page Faults = 12

===== Execution Successful =====

LRO:-

```

→ #include <stdio.h>
int main() {
    int pg, f;
    int n, i, j, k;
    int pgf = 0;
    int found, pos, back, index;

    printf("Enter no. of pg: ");
    scanf("%d", &n);

    printf("Enter pg reference string: ");
    for(i=0; i<n; i++) {
        scanf("%d", &pg[i]);
    }

    printf("Enter no. of frames: ");
    scanf("%d", &f);

    for(i=0; i<f; i++) {
        frames[i] = -1;
    }

    printf("OPR ");

    for(i=0; i<n; i++) {
        found = 0;
        for(j=0; j<f; j++) {
            if(frames[j] == pg[i]) {
                found = 1;
                break;
            }
        }
    }
}

```


64

```
if (found == 0) {
    pos = -1;
```

```
for (j = 0; j < n; j++) {
    if (frames[j] == -1) {
        pos = j;
        break;
    }
}
```

```
if (pos != -1) {
    frames = pages[i];
```

```
} else {
    farthest = -1;
    index = -1;
```

```
for (j = 0; j < n; j++) {
    if (frames[j] == -1) {
```

```
        for (k = i + 1; k < n; k++) {
            if (frames[k] == -1) {
                if (k > farthest) {
                    farthest = k;
                    index = j;
                }
            }
        }
    }
}
```

```
found where = 1;
break;
```

```
}
}
```

65

OPTIMAL PAGE REPLACEMENT
CODE:

```
#include <stdio.h>

int main() {

    printf("Name:Nimay Anand Desai\nUSN: 1WA24CS189\n\n");

    int pages[20], frames[10]; int n, f,
    i, j, k, pos, fault = 0, found;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string: ");
    for(i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++) {
        frames[i] = -1;
    }

    for(i = 0; i < n; i++) {

        found = 0;

        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
    }
}
```

```

if(found == 0) {

    pos = -1;

    for(j = 0; j < f; j++) {

        int future = 0;

        for(k = i + 1; k < n; k++) {
            if(frames[j] == pages[k]) {
                future = 1;
                break;
            }
        }

        if(future == 0)
            { pos = j;
              break;
            }
    }

    if(pos == -1)
        pos = 0;

    frames[pos] = pages[i]; fault+
    +;
}

printf("\nPage %d -> ", pages[i]);

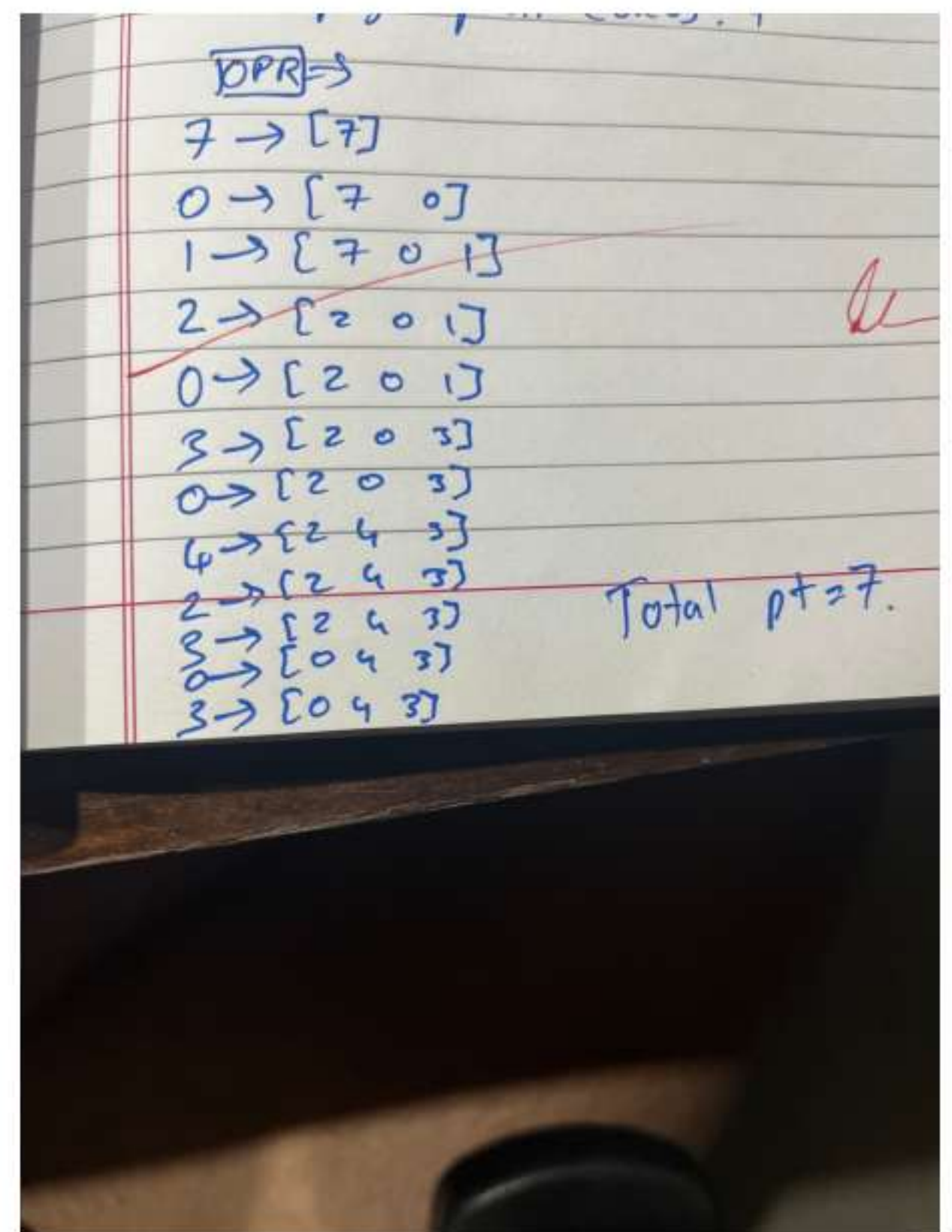
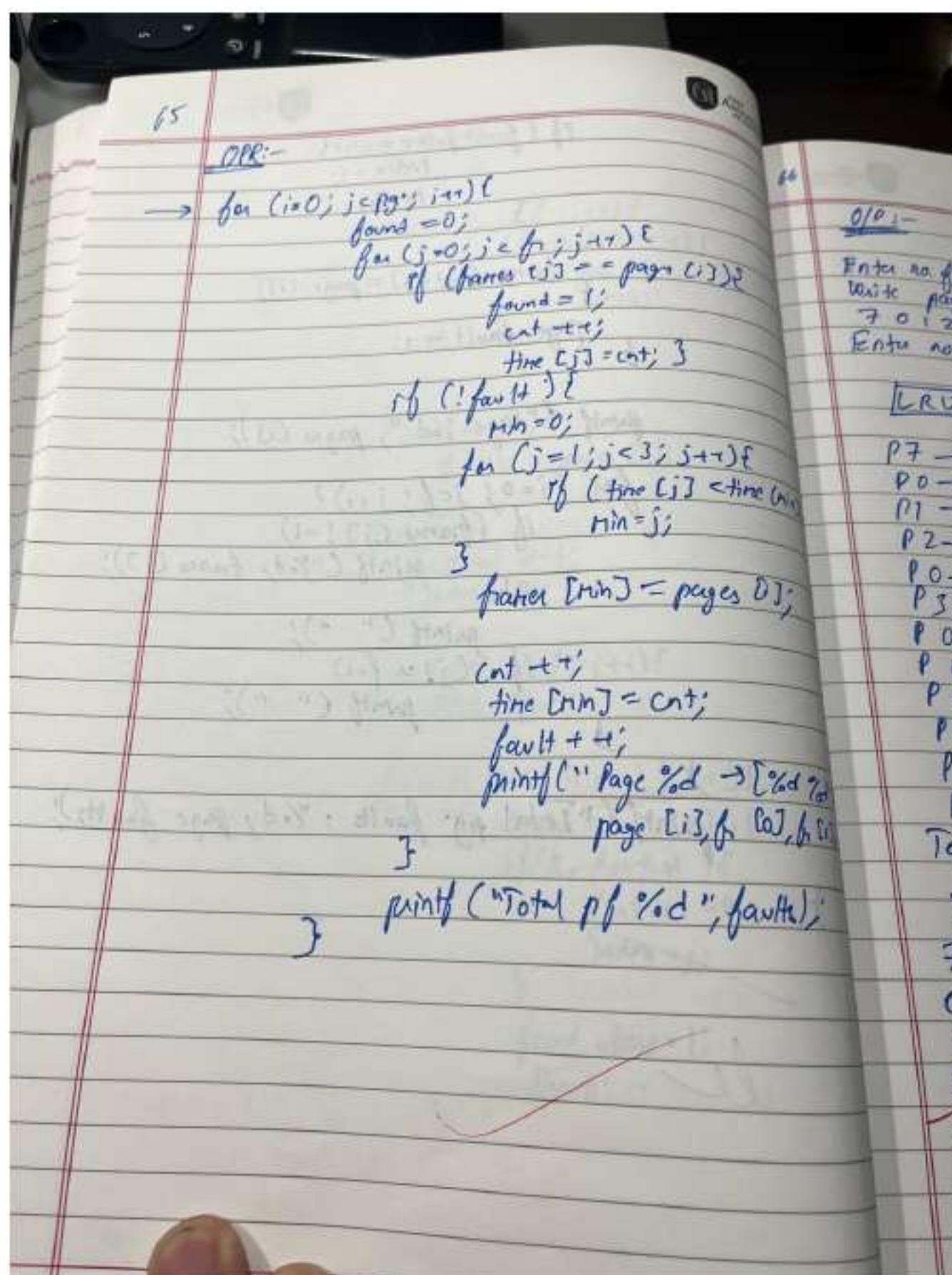
for(j = 0; j < f; j++) {
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}

printf("\n\nTotal Page Faults = %d\n",
        fault);return 0;

```


}

Result:



INDEX PAGE:-

OPERATING SYSTEM LAB				
INDEX OBSERVATION BOOK				
NAME		Npmay Anand Desai		
Sl. No.		Date		Class 4 th Sem Sec. 4-0 Admission No. 1WA24CS189
Sl. No.	Date	TITLE	Page No.	Teacher's Sign. Remark
1.		<u>lab 1</u> Program-1		
2.		FCFS	1	
3.		STF - Non-preemptive	2	
4.		STF - Pre-emptive	5	
			8	
5.		<u>lab 2</u> Non-preemptive priority schedu	10	
6.		Preemptive priority scheduling.	17	
7.		Round Robin	21	
8.		<u>lab 3</u> P8 - Multilevel Queue	24	
9.		P9 - Rate Monotonic Scheduling	26	
10.		P10 - Earliest Deadline First	28	
11.		P11 - Proportional Share Scheduling	31	
12.		<u>lab 4</u> Producer Consumer	35	
13.		Dining Philosophers	38	
15.		<u>lab 5</u> Banker's Algorithm	43	
16.		Dead lock detection algorithm.	47	
17.		<u>lab 6</u> P16 - First fit, best fit, worst fit.	54	
18.		<u>lab 7</u> P17 - FIFO	59	
19.		P18 - LRU Page Replacement		
20.		P19 - Optimal page replacement.		